

Aula 3 — Parte 2

Laboratório: Comparação Prática entre Aprendizado Supervisionado e Não Supervisionado

Data	Terça-feira, 18/08/2026 — sessão 6 de 36
Unidade da ementa	2. Paradigmas de aprendizado (aplicação prática)
Local	Laboratório de Informática (cada aluno em uma máquina)
Arquivos	lab03_parte2_demo.py • lab03_parte2_exercicios.py
Entrega	lab03_parte2_exercicios.py preenchido e executado + relatório de conclusão

Objetivos desta sessão

1. Aplicar **K-NN** (supervisionado) e **K-Means** (não supervisionado) no mesmo conjunto de dados e comparar os dois caminhos.
2. Perceber que o algoritmo não supervisionado **nunca usa y durante o treino** — comparar com y é só um recurso pedagógico, disponível apenas porque o dataset é didático.
3. Confirmar, em um segundo caso, que algoritmos baseados em distância — K-Means incluído — também exigem padronização.
4. Interpretar o **Adjusted Rand Index (ARI)** como medida de concordância entre agrupamento e rótulo real, sem confundi-lo com acurácia supervisionada.
5. Discutir os limites do não supervisionado: em um problema real, não existe y para comparar.

1. Retomada: dois caminhos, mesmo X

Na Parte 1 você treinou um K-NN (usando y) e um K-Means (sem usar y) no Iris, e comparou os grupos encontrados com as espécies reais só como verificação didática. Hoje repetimos essa comparação em um dataset onde a escala dos atributos já causou problema antes o Wine para confirmar que a lição da Aula 2 vale também para algoritmos não supervisionados.

2. O mesmo dataset, dois algoritmos

Relembrando da Aula 2: o Wine tem 178 amostras de vinho, 13 atributos químicos em escalas bem diferentes (magnésio na casa das dezenas, prolina na casa das centenas/milhares) e 3 cultivares como rótulo — mas hoje o rótulo só entra no caminho supervisionado.

2.1 Caminho supervisionado: K-NN

```
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

wine = load_wine()
```

```
X, y = wine.data, wine.target

X_treino, X_teste, y_treino, y_teste = train_test_split(
    X, y, test_size=0.30, random_state=42, stratify=y,
)
knn = make_pipeline(StandardScaler(), KNeighborsClassifier(n_neighbors=5))
knn.fit(X_treino, y_treino) # usa y para aprender
print("acuracia:", accuracy_score(y_teste, knn.predict(X_teste))) # usa y para avaliar
```

Trecho 1 — o padrão já conhecido da Aula 2, agora com padronização dentro do pipeline por hábito, já que sabemos que o Wine precisa.

2.2 Caminho não supervisionado: K-Means

Aqui não há treino/teste, porque não há y para avaliar do jeito supervisionado. O K-Means recebe todo o X e agrupa. `n_clusters` é um hiperparâmetro — assim como `k` no K-NN — só que agora ele representa o número de grupos, escolhido por quem modela.

```
from sklearn.cluster import KMeans
from sklearn.metrics import adjusted_rand_score
import pandas as pd

kmeans_sem_pad = KMeans(n_clusters=3, random_state=42, n_init=10)
clusters_sem_pad = kmeans_sem_pad.fit_predict(X) # SEM padronizar

scaler = StandardScaler().fit(X)
X_padronizado = scaler.transform(X)
kmeans_com_pad = KMeans(n_clusters=3, random_state=42, n_init=10)
clusters_com_pad = kmeans_com_pad.fit_predict(X_padronizado) # COM padronizacao

print("ARI sem padronizar:", adjusted_rand_score(y, clusters_sem_pad))
print("ARI com padronizar :", adjusted_rand_score(y, clusters_com_pad))
```

Trecho 2 — y só aparece na última linha, para medir o ARI. O KMeans em si nunca o recebe.

Leitura esperada

Sem padronizar, o ARI fica bem baixo — a prolina, na casa das centenas/milhares, domina a distância euclidiana do K-Means exatamente como dominava a do K-NN na Aula 2, e os grupos encontrados quase não correspondem aos cultivares reais. Com padronização, o ARI sobe de forma acentuada. A mesma causa (distância dominada por uma coluna) e o mesmo remédio (StandardScaler) valem para os dois algoritmos, supervisionado ou não — porque o problema está na métrica de distância, não no uso de y.

3. O que a comparação NÃO pode ser

Cuidado conceitual

É tentador dizer "o K-Means acertou X% dos cultivares", como se fosse uma acurácia. Isso é impreciso: o K-Means não tenta prever o rótulo 0, 1 ou 2 — ele apenas separa em grupos, e o número do grupo é arbitrário (o grupo "0" do algoritmo pode corresponder ao cultivar "2"). O ARI resolve isso comparando a estrutura dos agrupamentos, não os números literais dos rótulos. Nunca use `accuracy_score` entre y e clusters — o resultado seria enganoso.

4. Os limites do não supervisionado

Tudo isso só foi possível porque o Wine é um dataset didático, com y conhecido. Em uma aplicação real de agrupamento — segmentar clientes, agrupar documentos, encontrar padrões de uso — você não tem esse y para calcular ARI. A pergunta "o agrupamento encontrado é bom?" precisa de outro tipo de resposta, que não depende de rótulo: veremos isso formalmente na Aula 10 (métricas de avaliação de agrupamentos, como a silhueta).

O que guardar desta sessão

- K-NN usa y para treinar e avaliar; K-Means nunca usa y — comparar com y é recurso didático, não parte do algoritmo.
- Algoritmos baseados em distância (K-NN, K-Means, e outros que virão) são sensíveis à escala, sempre pelo mesmo motivo: a distância euclidiana é dominada pela coluna de maior amplitude.
- ARI mede concordância estrutural entre agrupamento e rótulo; não é o mesmo que acurácia, e `accuracy_score` entre y e clusters não deve ser usado.
- Em problemas não supervisionados reais, não há y para comparar — a avaliação de agrupamento (Aula 10) precisa de outros critérios.